ALBUKHARY INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATICS (SCI)

Bachelor of Data Science

COURSE NAME: PROGRAMMING AND PROBLEM SOLVING

COURSE CODE: CCC1123

GROUP PROJECT REPORT

GROUP NAME: CODE & COOK CREW

Restaurant Ordering System

GROUP: C2_BDS1A

LECTURER NAME:

DR. BAHARUDDIN OSMAN & MR. ABU BAKAR NGAH

No.	Name	Student ID
1.	Hamdha Banu Ainudeen Hasiyar	AIU25102230
2.	Ismail Hosen	AIU25102207
3.	Mohamed Armash Mohamed Rifkan	AIU25102232
4.	Phyo Wai Aung	AIU25102210
5.	Thahida Mohd Noor	AIU25102211

INTRODUCTION

This project focuses on developing a simple C-based restaurant food ordering system to assist in managing customer orders efficiently. In many restaurants, especially during peak hours, staff often handle multiple orders at the same time, which can lead to mistakes such as missing items or incorrect order records when using manual methods.

The proposed system allows users to select food, drinks, and sides from a displayed menu, enter quantities, and automatically calculate the total price. Additional features such as service charge and takeaway charge are included to reflect real-world restaurant operations. By using a computerized approach, the system improves accuracy, reduces human error, and speeds up the ordering process, resulting in better service efficiency and customer satisfaction.

PROBLEM STATEMENT

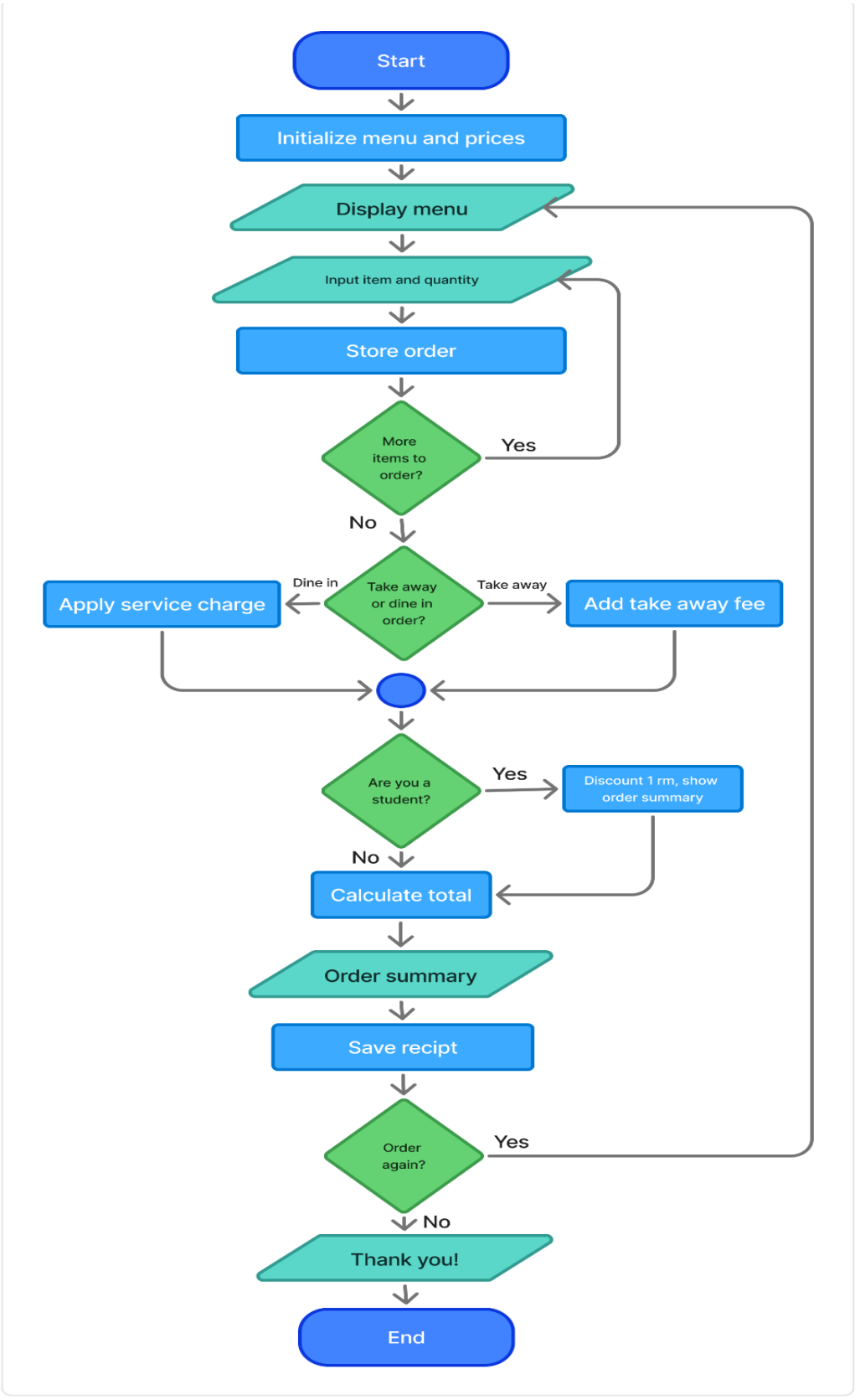
Manual food ordering methods used in many restaurants are prone to errors such as incorrect orders, missing items, and inaccurate billing, especially during busy periods. These issues can lead to delays and customer dissatisfaction.

Therefore, a computerized food ordering system is needed to improve order accuracy, speed up the ordering process, and ensure correct bill calculation.

PROJECT OBJECTIVES

1. To identify the inputs, processes, and outputs required for a C-based food ordering system.
2. To design a simple algorithm and program structure using arrays, functions, and conditional statements.
3. To implement the food ordering system using the C programming language with appropriate programming constructs.
4. To test the system to ensure accurate order processing and correct total price calculation.

FLOW CHART


Start
Initialize menu and prices
Display menu
Input item and quantity
Store order
More items to order?
Yes
No
Dine in
Take away or dine in order?
Take away
Apply service charge
Add take away fee
Are you a student?
Yes
Discount 1 rm, show order summary
No
Calculate total
Order summary
Save recipt
Order again?
Yes
No
Thank you!
End

Before running the program, please download our sourse code (.c source file) for our restaurant management system.

Click here to download.

CODE

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
// Global variable for service charge rate
float serviceChargeRate = 0.05;

// Function prototypes
int loadMenu(char menu[][20], float prices[], int maxItems);
void takeOrder(char menu[][20], int menuSize, char orderedItems[][20], int orderedQuantities[],
               int *totalOrders, float prices[], float *totalPrice);
void showOrder(char orderedItems[][20], int orderedQuantities[], int totalOrders, char
               menu[][20], float prices[], int menuSize);
float applyServiceCharge(float amount);
void printTotalItems(int total);
void saveReceipt(char orderedItems[][20], int orderedQuantities[], int totalOrders,
              char menu[][20], float prices[], int menuSize, float finalTotal);

int main() {
    char menu[50][20];
    float prices[50];

    // Load menu
```

```
int menuSize = loadMenu(menu, prices, 50);
if (menuSize == 0) {
    printf("No menu loaded. Exiting program.\n");
    return 1;
}

int running = 1;
while (running) {
    // Display menu
    printf("\n--- MENU ---\n");
    for (int i = 0; i < menuSize; i++) {
        printf("%s @ RM%.2f\n", menu[i], prices[i]);
    }

    // Order variables
    char orderedItems[20][20];
    int orderedQuantities[20];
    int totalOrders = 0;
    float totalPrice = 0.0;
    int isTakeaway, isStudent;

    // Take order
    takeOrder(menu, menuSize, orderedItems, orderedQuantities,
            &totalOrders, prices, &totalPrice);

    if (totalOrders == 0) {
        printf("No items ordered.\n");
    } else {
        // Takeaway or dine-in
```

```
printf("\nIs this order takeaway? (1 = Yes, 0 = Dine-in): ");
scanf("%d", &isTakeaway);

if (isTakeaway) {
   float takeawayCharge = 0.50 * totalOrders;
   totalPrice += takeawayCharge;
   printf("Takeaway charge RM%.2f added.\n", takeawayCharge);
} else {
   totalPrice = applyServiceCharge(totalPrice);
   printf("5%% service charge applied.\n");
}

// Student discount
printf("\nAre you a student? (1 = Yes, 0 = No): ");
scanf("%d", &isStudent);

if (isStudent && totalPrice >= 1.00) {
   totalPrice -= 1.00;
   printf("Student discount RM1.00 applied.\n");
}

// Show summary
showOrder(orderedItems, orderedQuantities,
        totalOrders, menu, prices, menuSize);

// Save receipt
saveReceipt(orderedItems, orderedQuantities,
          totalOrders, menu, prices, menuSize, totalPrice);
```

```
            printf("\nFinal Total: RM%.2f\n", totalPrice);
            printTotalItems(totalOrders);
        }

        // Order again?
        int choice;
        printf("\nWhat would you like to do next?\n");
        printf("1. Order Again\n2. Exit\nChoose option: ");
        scanf("%d", &choice);

        if (choice == 2) {
            running = 0;
            printf("\nThank you for visiting!\n");
        }
    }
    return 0;
}

// Load menu from file
int loadMenu(char menu[][20], float prices[], int maxItems) {
    FILE *file = fopen("C:\\Users\\DELL\\Downloads\\menu.txt", "r");
    if (!file) {
        printf("Error: Could not open menu file.\n");
        return 0;
    }

    int count = 0;
    char name[20];
    float price;
```

```
    while (fscanf(file, "%s %f", name, &price) == 2 && count < maxItems) {
        strcpy(menu[count], name);
        prices[count] = price;
        count++;
    }

    fclose(file);
    return count;
}

// Take order
void takeOrder(char menu[][20], int menuSize,
               char orderedItems[][20], int orderedQuantities[],
               int *totalOrders, float prices[], float *totalPrice) {

    char item[20];
    int quantity;
    int choice;

    printf("\nStart ordering.\n");

    while (1) {
        printf("\nEnter item name: ");
        scanf("%s", item);

        // Convert input to uppercase
        for (int i = 0; item[i]; i++) {
            item[i] = toupper(item[i]);
```

```
}

int found = 0;

// Check if item exists in menu
for (int i = 0; i < menuSize; i++) {
    if (strcmp(item, menu[i]) == 0) {
        found = 1;

        printf("Enter quantity: ");
        scanf("%d", &quantity);

        strcpy(orderedItems[*totalOrders], item);
        orderedQuantities[*totalOrders] = quantity;
        *totalPrice += prices[i] * quantity;
        (*totalOrders)++;
        break;
    }
}

//Message if the item is not found
if (!found) {
    printf("Item not found in menu. Try again.\n");
    continue;
}

// Ask if user wants to order more
printf("\nWould you like to order more?\n");
printf("1. Yes\n2. No\nChoose option: ");
```

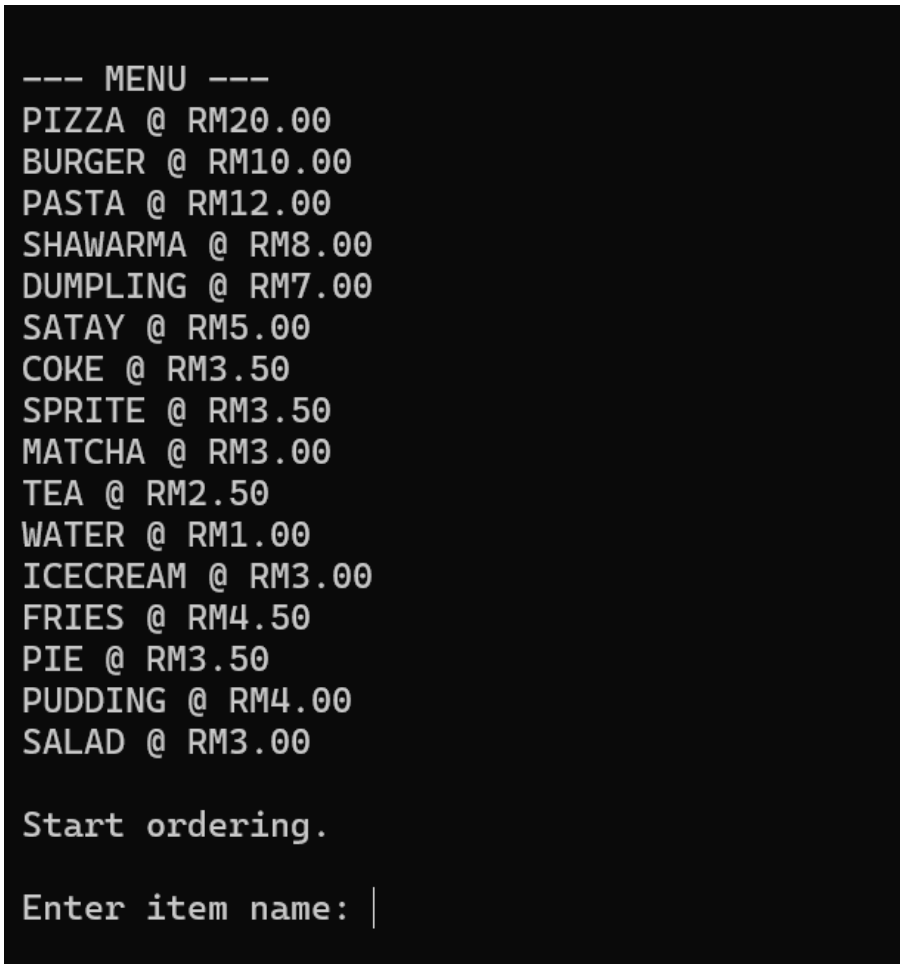
```
        scanf("%d", &choice);

        if (choice == 2) {
            break;
        }
    }
}

// Show order summary
void showOrder(char orderedItems[][20], int orderedQuantities[],
               int totalOrders, char menu[][20], float prices[], int menuSize) {

    printf("\n--- Your Order Summary ---\n");
    for (int i = 0; i < totalOrders; i++) {
        for (int j = 0; j < menuSize; j++) {
            if (strcmp(orderedItems[i], menu[j]) == 0) {
                printf("%d x %s @ RM%.2f = RM%.2f\n",
                       orderedQuantities[i],
                       orderedItems[i],
                       prices[j],
                       prices[j] * orderedQuantities[i]);
                break;
            }
        }
    }
}

// Apply service charge
float applyServiceCharge(float amount) {
```

```
    return amount + (amount * serviceChargeRate);
}

// Print total items
void printTotalItems(int total) {
    printf("You ordered %d different items.\n", total);
}

// Save receipt
void saveReceipt(char orderedItems[][20], int orderedQuantities[],
                 int totalOrders, char menu[][20], float prices[],
                 int menuSize, float finalTotal) {

    FILE *file = fopen("receipt.txt", "a");
    if (!file) {
        printf("Error: Could not create receipt file.\n");
        return;
    }

    fprintf(file, "\n--- RECEIPT ---\n");

    for (int i = 0; i < totalOrders; i++) {
        for (int j = 0; j < menuSize; j++) {
            if (strcmp(orderedItems[i], menu[j]) == 0) {
                fprintf(file, "%d x %s @ RM%.2f = RM%.2f\n",
                        orderedQuantities[i],
                        orderedItems[i],
                        prices[j],
                        prices[j] * orderedQuantities[i]);
```

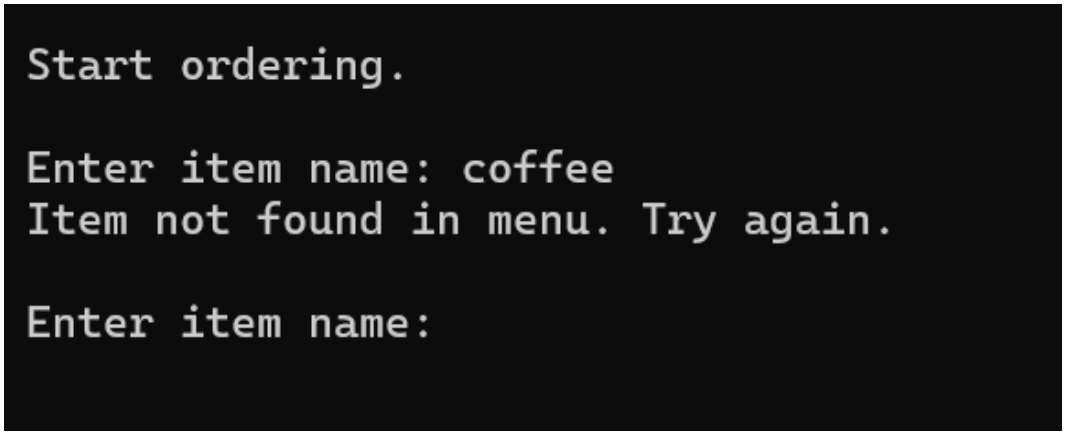
```
            break;
        }
    }
}

fprintf(file, "Final Total: RM%.2f\n", finalTotal);
fprintf(file, "----------------------------\n");

fclose(file);

printf("\nReceipt saved to receipt.txt\n");
}
```

OUTPUT

1. This is the menu display we can see when we run the program. It is saved in the menu.txt file.



2. If we order an item which is not in the menu, It will appear like this.



3. After each order item and quantity, a question is asked to customers to order more or not. If they want to order more, new item names and quantities will be asked from the customer.

```
Enter item name: pizza
Enter quantity: 2

Would you like to order more?
1. Yes
2. No
Choose option:
```

4. If they do not want to order more, a question will be asked whether they want to takeaway or dine-in. If they want to dine-in, a 5% service charge will be applied to their total order.

```
Would you like to order more?
1. Yes
2. No
Choose option: 2

Is this order takeaway? (1 = Yes, 0 = Dine-in): 0
5% service charge applied.
```

5. If the customer wants to takeaway, a takeaway charge RM 0.50 will be added.

```
Is this order takeaway? (1 = Yes, 0 = Dine-in): 1
Takeaway charge RM0.50 added.
```

6. Then, we ask the customer if they are a student or not. If yes, a student discount RM1.00 is applied. If not, there will not be any changes in total cost.

```
Are you a student? (1 = Yes, 0 = No): 1
Student discount RM1.00 applied.
```

7. After that, we will show the order summary, including Final cost, ordered items, and the receipt is saved into the receipt.txt file.

```
--- Your Order Summary ---
2 x PIZZA @ RM20.00 = RM40.00
2 x PUDDING @ RM4.00 = RM8.00
2 x WATER @ RM1.00 = RM2.00

Receipt saved to receipt.txt

Final Total: RM51.50
You ordered 3 different items.
```

8. Finally, we ask the customer if they want to order again or exist. If they choose to order again, the menu will be displayed again and the program will be looped. If the customer chooses to exist, the program will end by displaying “Thank you for visiting!” message.

```
What would you like to do next?
1. Order Again
2. Exit
Choose option: 2

Thank you for visiting!
```